

28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)

LLM-based methods for the creation of unit tests in game development

Ciprian Paduraru^a, Adelina Staicu^a, Alin Stefanescu^{a,b}

^aUniversity of Bucharest, Department of Computer Science, Romania

^bInstitute of Logic and Data Science, Romania

Abstract

Problems related to the quality of games, whether on the initial release or after updates, can lead to player dissatisfaction, media attention, and potential financial setbacks. These issues can stem from software bugs, performance bottlenecks, or security vulnerabilities. Despite these challenges, game developers often rely on manual playtesting, highlighting the need for more robust and automated processes in game development. This research explores the application of Large Language Models (LLMs) to automate the creation of unit tests in game development, focusing on strongly typed programming languages such as C++ and C#, which are widely used in the industry. The study focuses on fine-tuning Code Llama, an advanced code generation model, to address common scenarios in game development, including game engines and specific APIs or backends. Although the prototyping and evaluations primarily took place within the Unity game engine, the proposed methods can be adapted to other internal or publicly available solutions. The evaluation results demonstrate these methods' effectiveness in improving existing unit test suites or automatically generating new tests based on natural language descriptions of class contexts and targeted methods.

© 2024 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems

Keywords: Large language models; fine-tuning; game development; unit tests;

1. Introduction

Ensuring the quality of video games and integrating effective unit testing into the game development process presents several challenges. Research studies, including the one cited by [20], consistently show that games, regardless of budget or team size, often come to market with defects. These issues can lead to heavy criticism and potential financial setbacks if the game fails to meet user expectations. A significant portion of these quality issues can be attributed to inadequate testing procedures. Surprisingly, game developers often rely on manual game testing and the experience of human testers. Although automation is recognized as a crucial step in improving game quality, existing testing procedures are not always applicable to all types of games. In some high-profile projects, developers even work without a single unit test. In addition, most developers specialize in specific areas of a project, highlighting the need for more robust and automated

testing across the entire game development lifecycle. However, the unique characteristics of games — such as interactivity and immersion — pose a challenge when implementing traditional software testing methods.

With the increasing importance of Large Language Models (LLMs) in code generation, researchers have investigated their accuracy in code generation [14], code quality [23], impact on security [19], and their potential for API learning and code complexity prediction. How this can be applied to game and engine development is an important and interesting problem. Challenges arise from the limited availability of training data, strict type checking in strongly typed languages, and differences in the APIs and terminologies used. This study explores the possibilities of LLMs to automate the creation of unit tests for game development. We start with Code Llama [21] as a base model and refine it using a dataset drawn from game development examples.

Our contribution can be summarized as follows:

- To the best of our knowledge, this study represents the first investigation of the use of LLMs to create unit tests specifically for game development, independent of the game engine, considering two commonly used statically typed languages: C# and C++.
- We present an automatic method for creating corpora based on LLMs to generate synthetic datasets of unit tests for training models or evaluation purposes. The goal is to remove the bottleneck of not having enough data available or the cost of acquiring data by human experts.
- The objectives of the work are defined after discussions with the local game development industry. The experiments are conducted with two publicly available game engines, Unity¹ and Unreal², which are used by companies or have the most in common with private engines in companies.
- Our work includes the adaptation of LLM-related processes, such as fine-tuning, prompting, and retrieval augmented generation (RAG) [15], to typical game development scenarios.
- Our prototype is available on demand at <https://github.com/unibuc-cs/GameTestingWithLLMs>

The following sections of this article are organized as follows. First, we discuss related work that is in line with our research goals. In Section 3, we elaborate on the process of extracting a dataset specifically related to game development source code. Section 4 describes the methods and procedures used for sampling, filtering, and fine-tuning the LLM model. The results of our experiments, limitations, and possible improvements are presented in the Evaluation section. Finally, we draw our conclusions in the last section.

2. Related work

Difficulties with unit testing. The process of manually developing and maintaining unit tests can be a daunting and tedious task [25]. Researchers have proposed various methods for automatically generating unit tests. Traditional methods typically use search-based [4], constraint-based [1] or random-based strategies [5], [6]. The main goal of these techniques is to create a suite of unit tests that optimize the coverage of the software under test. Although these automatically generated tests achieve a satisfactory level of coverage, they are often less readable and meaningful compared to manually written tests. Therefore, developers are usually reluctant to integrate them directly into their workflow [25].

LLMs in Code Generation. Numerous language learning models such as CodeBert [8], CodeGen [17], CodeT5 [27], Llama2-Code [21] have been made available for code generation after being fine-tuned on large code datasets. Subsequently, GitHub Copilot has introduced an advanced autocomplete feature that helps with basic algorithmic problems and uses an improved version of Codex [7]. Our research does not focus on code generation, but rather on how LLMs can be fine-tuned and customized for generating unit tests using a zero-shot prompt, with the potential to incorporate existing codebases of game development APIs and infrastructures.

¹ www.unity.com

² www.unrealengine.com

LLMs in Unit Test Generation. The study in [28] is an empirical evaluation of the ability of Large Language Models (LLMs) to generate unit tests without additional training or manual intervention for Java code, comparing the results of three public models: Codex[7], GPT-3.5-Turbo, and StarCoder [11]. The authors first examine the impact of the context provided as a prompt on the process of generating unit tests. The models are evaluated using various metrics: compilation rates, test accuracy, test coverage and test smell. A similar goal is outlined in [24], where the authors evaluate ChatGPT, this time to evaluate the generated unit tests. Our approach differs from their work in that we implement a fine-tuning method to adapt the LLM base model to capture the code of the basic functions of the game engine. We then compare the unit tests generated by the baseline LLM models and the fine-tuned models using the same metrics. The TestGen-LLM tool developed by Meta [2] uses the Code Llama model [21] to improve existing tests created by software engineers. This tool automatically refines the test cases and validates the improved versions using a series of filters to ensure a significant improvement over the original test suite. It also addresses issues related to LLM hallucinations. TestGen-LLM was effectively implemented on Meta’s platforms, including Instagram and Facebook, where it increased the quality of existing unit test sets by 11.5%, according to the report. Furthermore, a significant proportion of these generated tests, 73%, were approved for production by Meta’s software engineers. In contrast, our preliminary goal is to adapt the unit test generation models for game-specific projects and align the evaluation purpose with the ability to generate new tests from scratch when only a limited number of examples are available as a training resource, as is typically the case for game project databases that do not have extensive unit test sets that can be further improved. The work is also interesting for our use case as their method could be used to improve the automatically generated tests through a feedback improvement loop. This is an area for our future work.

3. Methods for collecting a custom corpus

The motivation to create a training and evaluation corpus as part of this project stems from two main observations made after an initial investigation of existing datasets of unit tests:

1. Limited resources for languages such as C++ and C#.
2. The lack of datasets for game development. This is an important aspect because sometimes the terminology and APIs used can differ from the general programming cases.

The special thing about game development is that the source code often depends on the components of the base engine (e.g. those in Unity on the Unreal Engine) or on project-internal libraries and APIs. During testing, these components are mocked so that the tests can be executed independently of each other, as required by the standard unit testing methodology. Both the training and evaluation corpora should prepare (fine-tune) the LLM core to understand these backend functions, their components, classes, methods and interfaces.

Our method first attempts to collect unit tests from publicly available projects. Using this dataset, which is denoted as $\mathcal{D}_h$, a preliminary model called *GameUnitLLM_{v0}* is trained to generate unit tests. Based on this first version, a synthetic method to automatically create a set of valid unit tests without human annotation. This dataset is named $\mathcal{D}_s$, thus that the total corpora is represented by the union of the two: $\mathcal{D} = \mathcal{D}_h \cup \mathcal{D}_s$. This is further divided into two subsets, a training set and a test set, i.e. $\mathcal{D} = \mathcal{D}_{test} \cup \mathcal{D}_{train}$.

For evaluation purposes and comparisons between the two programming languages typically used in game development, we split these quantities for unit tests into C# and C++ respectively: $\mathcal{D}^{cs}$ and $\mathcal{D}^{cpp}$. The fine-tuning of $\mathcal{D}_{train}$ results in the final LLM model *GameUnitLLM*. A separate fine-tuning is performed using the two splits, resulting in the models *GameUnitLLM^{cs}* and *GameUnitLLM^{cpp}* respectively. The two steps are explained later in this section and can be visualized in Figure 1.

3.1. Dataset of unit tests written by experts

First, nine project implementations that have been made available as open source are collected as follows:

- Five projects from the Unity learning platform³ and their corresponding task solutions examples from GitHub repositories.
- The four (at the time of writing) educational game examples from the Unreal Engine on their marketplace platform⁴.

From these projects, we identify the first layer of dependencies and extract their source code, i.e. a collection of C++ and C# script files whose source code was publicly available. The extracted packages include functions and APIs specifically for game development such as animation, UI-related packages, pathfinding, physics, input processing, etc.

The source code corresponding to the unit tests is automatically extracted from the collected projects by searching through the scripts and picking out the methods that have a specific attribute according to the Unity/Unreal backend implementation (e.g. `[TEST]` in Unity or `[...AUTOMATION__TEST]` in Unreal). These methods are then grouped by class and we compile all the available code for the class definition (*Cls*) together with the corresponding discovered unit tests, i.e. pairs of the methods under test (*MUTs*) and their respective unit test source code. The data set obtained from experts writing the source code is called $\mathcal{D}_h$. It is grouped by class and method tested, as shown in Eq.(1), where each $D_i \in \mathcal{D}_h$ consists of the following elements:

- (a) A parent class: Cls_i ,
- (b) The set of methods to be tested in this class: $MUT_j \in Cls_i$.
- (c) The suite of unit tests for each MUT_j : $UT(MUT_j)_k$ (filtered, only those with status *passing* to ensure the correctness of the selected data).

$$\mathcal{D}_h = (Cls_i, MUT_j, UT(MUT_j)_k)_i \quad (1)$$

3.2. Synthetic dataset

The process of collecting supervised data, either from human annotators or via human feedback, as described in [18], can be costly in programming tasks as it requires the involvement of professional developers. Instead of human feedback, our proposed methods use an execution feedback method [2] to create artificial data with unit tests to fine-tune the generative models. Using the $\mathcal{D}_h$ dataset, the first version of the fine-tuned unit test code generation model, *GameUnitLLM_{v0}*, is created. This model is further used to generate correct synthetic unit tests without expert annotations by passing it to the function shown in Listing 1. First, one class and one method are sampled equally, line 2. Note that this strategy can also be used to test new functions outside the original set. The metadata of the selected class and function is first passed to a prompt template and then through the model to generate a new unit test U , line 4 (see Section 4.1 for detailed explanation). The resulting code is first compiled, line 5. When it is compiled, a build is performed and the correctness of the test is verified by checking its *pass* status. These two checks are necessary to ensure that the model can learn clean (correct) data when the test is added to the dataset. Finally, a utility check is performed: checking the coverage of U compared to the coverage of the original dataset (e.g. are new lines or branch points covered?). If the new test is useful, it is added to the set, line 6. The process based on the pipeline is also shown in Figure 2.

³ <http://learn.unity.com>

⁴ <https://www.unrealengine.com/marketplace/en-US/store>

```

1 GenerateUnitTest (LLM) :
2   Sample a class  $Cls_i \in \mathcal{D}$  and a method to test  $MUT_j \in Cls_i$ 
3   # Pass through the model the metadata for parent class and function to test
4    $U = LLM(Prompt_{gen}(\mathcal{M}(Cls_i), \mathcal{M}(MUT_j)))$ 
5   if  $COMPILES(U)$  and  $test\_pass(EXECUTE(U))$  and  $COV(U) \notin COV(\mathcal{D}_h)$  then:
6      $\mathcal{D}_s = \mathcal{D}_s \cup (Cls_i, MUT_j, U)$ 

```

Listing 1: Steps for creating a synthetic dataset

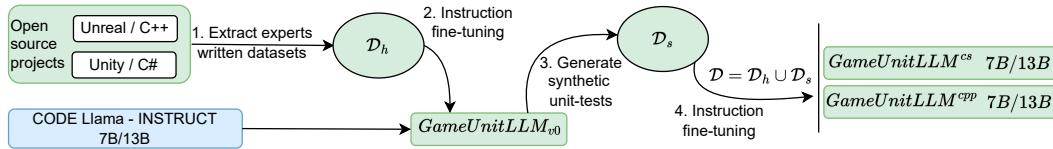


Fig. 1: The refinement of the generative unit test model *GameUnitLLM* starts with the base models of Code Llama [21] Instruct 7B and 13B, represented by the blue rectangle. fine-tuning is then performed with the extracted and processed data sets, shown by green rectangles). First, the already written expert unit tests that are considered to work are collected, $\mathcal{D}_h$. On this basis, a fine-tuned version of the generative unit test model, *GameUnitLLM_{v0}*, is created. Using this model, a synthetic dataset $\mathcal{D}_h$ is created without any human effort. Finally, using the combined datasets, the same fine-tuning method generates the final generative models, one each for C++ and C#.

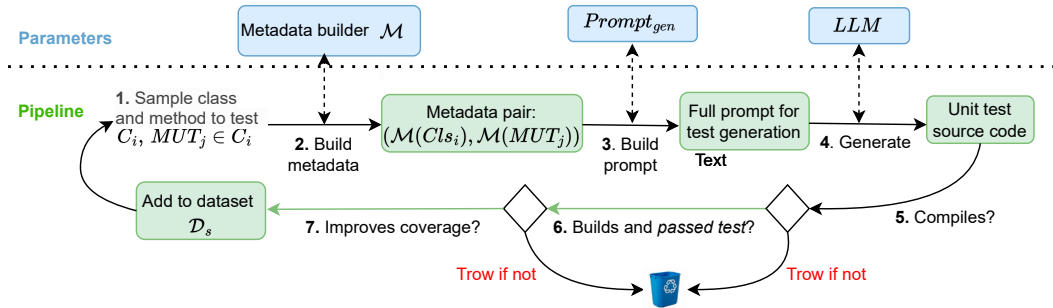


Fig. 2: The pipeline for generating a batch of unit tests given the instances of different components as parameters. The main parameters (top row) are, from left to right: P1-the metadata data builder given a class and function, P2-the prompt template used to generate the test based on metadata, P3-the LLM instance used to take as input the prompt and generate one or more unit tests. The bottom row represents the continuous pipeline used to generate the unit tests, within 7 steps. The output produced by some individual steps is shown with green boxes. Generated tests are filtered before adding them to data synthetically generated datasets. The process checks if they compile, build as passed, and improve existing coverage.

3.3. Dataset numbers

The first step in the dataset collection process as mentioned in Section 3 captured human written unit-tests collected from 3700 C++ and C# script files, 2941 classes, 319 MUTs and 726 corresponding unit tests. Out of these, 191 MUTs and 286 unit-tests were made for Unity and C# language. Furthermore, the split between training and evaluation is made by the 80/20 rule. The synthetic dataset was produced using the algorithm in Listing 1 by letting a server of 20 high-performance CPUs compiling and executing instances of unit-tests for 7 days, 168h. While the compilation effort was not significant, execution time took between 1 second to 11 minutes. The main observation at this preliminary step is that in general, the model has more issues in producing valid C++ tests than using C#. Table 1, shows the final dataset grouped by the sizes of each split.

```

1 Generate  $\{N\}$  unit tests for the function in variable  $\{FUNC\}$  using language in
  variable  $\{LANGUAGE\}$  and the metadata given below:\n\n
2 # The parent class interface is in variable  $\{PARENT\_CLASS\}$  \n\n %
3 # The available mock classes/functions interfaces are defined in variable
   $\{MOCK\_CLASSES\}$  \n\n
4 All tests should be enclosed using a  $\langle GameUnitLLM \rangle \langle /GameUnitLLM \rangle$  tag:
5 1. Include the same required packages or files for types as in  $\{FUNC\}$  and any
  additional.\n
6 2. If you need interfaces that are not defined write each description in a json output
7 3. Always assert statements at the end of the test.

```

Listing 2: The general template for the prompt used to create tests. The variables between curly brackets are placeholders that are filled at the time of training or testing

4. Methods

4.1. Base Model and refining

In this study, we utilize the Code Llama [21] models, which are built on the foundation of Llama 2, a state-of-the-art open-source model at the time of writing. Code Llama models are optimized for code generation, using self-attention mechanisms to comprehend relationships and dependencies within the code. Specifically, we start with the Code Llama - Instruct class, which is fine-tuned to interpret natural language coding instructions. The models can take up to 10^5 tokens of context (note that typical LLMs can take up 4096 tokens), which is an important feature since developers can provide more context from their source code. This is needed since the dataset $\mathcal{D}$, from our observations, contains many classes that exceed 5×10^3 tokens.

The foundation model's fine-tuning is necessary to comprehend the game engine-specific functions, their components, classes, methods, and interfaces in general. The dataset $\mathcal{D}$, as defined in Eq. (1), undergoes additional processing through a series of steps outlined below. The objective is to automatically modify the source code instances and utilize them as an instruction refinement dataset for the base models. The entire process from the base model to the refined version suggested by our techniques, *GameUnitLLM*, is depicted in Figure 1. The refinement techniques and the source code in the repository are not tied to the Code Llama Instruct versions, meaning the techniques are compatible with model versions 7B, 13B, 34B, and 70B, where B represents billions of parameters.

During both training and testing phases, the LLM is instructed to generate tests using the prompt template shown in Listing 2. The primary concept is to design a reusable template with placeholder variables that can be populated as required during runtime. For instance, during the training phase, a dataset (explained below) is utilized to sample instances of inputs and versions of the anticipated outputs that the model should ideally generate. It's important to note that during the testing phase, to enhance user accessibility and in line with the methods proposed in [21], the user instructs the model to generate unit tests for a specific function and parent class using natural language. Automated scripts extract the necessary information from this request and populate the required placeholder variables in the prompt instance for user convenience.

For the training phase, the dataset $\mathcal{D}$ (Eq. (1)) is augmented for instruction fine-tuning, similar to the dataset utilized during the training of Code Llama [21]. Automated scripts are devised to extract the necessary metadata for each available unit test source code using Python scripts for both C++, using *clang*⁵, while for C# its native reflection support is supported⁶. This extraction procedure identifies the context of the parent class, Cls_i , and the context of the method under test, i.e., its source code and signature, $MUT_j \in Cls_i$. The result is denoted as $M_{i,j} = Metadata(Cls_i, MUT_j)$. This is subsequently used to

⁵ <https://pypi.org/project/libclang>

⁶ <https://github.com/pythonnet/>

populate the placeholder variables in the prompt outlined in Listing 2, i.e., $Prompt[M_{i,j}]$. The concept of populating templates using metadata from existing code bases is a general concept known as Retrieval Augmented Generation (RAG) [3].

$$D_{i,j,k}^{train} = \{Input : Prompt[M_{i,j}] Answer : (MUT_j)_k\} \quad (2)$$

Eq. 2 illustrates the technical structure of the instruction dataset used to refine the *GameUnitLLM* model from the foundational Code Llama model. Each entry consists of a completed prompt, which serves as input to the model, and a corresponding ground truth test (deemed correct, i.e., *passing* evaluation) that the refined version is expected to produce. As indicated in Listing 3 line 7, the cross entropy [10] is utilized as a loss function to enhance the model's performance during the training phase by passing the input fields of entries sampled from the dataset D^{train} and comparing them with the corresponding (ground truth) test code for similarity. To assess the model's performance at the testing phase, the perplexity metric [13] is favored.

```

1 for each epoch :
2   for each step in epoch :
3     Sample batch of examples  $\mathcal{B} \sim D^{train}$ 
4     # Pass through the model the inputs from  $\mathcal{B}$  to obtain predicted answers
5      $PredAnswers = GameUnitLLM(\mathcal{B}["Input"])$ 
6     # Compute loss for optimization process by comparing how close is the output of
7     the model compared to the example test.
8      $Loss = CrossEntropy(PredAnswers, \mathcal{B}["Answer"])$ 

```

Listing 3: Fine-tuning process of the two GameUnitGen models.

Following the generation process, both during training and inference, the produced tests undergo post-processing to automatically enhance their accuracy before they are ultimately output. Specifically, each generated unit test is initially compiled with the parent class and the necessary libraries. At this stage, the majority of errors are due to syntax issues, such as: test code including natural language explanations without comments, duplication of the class/functions in the prompt, minor alteration of imported packages, generation of values for local variables beyond the range specified in their data type, or generation of incomplete mock classes. To address these commonly observed problems, our approach is to use a few heuristics to enhance the results accuracy when the issue is basic and does not requires human input.

5. Evaluation

5.1. Details of the fine-tuning and evaluation setup.

A fine-tuning procedure, as described in Section 4.1, is used to customize the Code Llama Instruct-7B and 13B models using a dataset specific to the two game engines' APIs and several individual projects based on them. The result of this process is two models: GameUnitGen-7B and 13B respectively. Extending the capacity with the larger model Code Llama Instruct - 70B could bring further improvements, but would require significantly more computing resources and training time. The process was performed in $\sim 3500 GPU-hours$ using an Nvidia DGX V100 system with 8 GPUs with 32 GB RAM each. The fine-tuning method used is LoRA (Low-Rank Adaptation) [9], which can add updated weights to each selected layer of the base model with manageable computational effort. All layers were fine-tuned with rank $r = 16$, $\alpha = 32$, $dropout = 0.05$. The default quantization parameters with *bfloat16* were used to optimize the runtime of the training. For decoding, we used nucleus sampling with $p = 0.95$ and $temperature = 0.8$. However, during test time, the temperature was set to zero to achieve deterministic results that are predictable and repeatable, a decision inspired by previous research [23], [2]. The AdamW optimizer [12] is used, with $\beta_1 = 0.9$ and $\beta_2 = 0.95$, with a learning rate $\gamma = 3 \times 10^{-3}$, and a decay rate of 0.01. In the first training phase, a warm-up phase of 200 steps was used. Each batch had a size of 32, with 4 gradient accumulation steps and different

Table 1: The size of the data sets is divided into categories that are used for the training and evaluation process. For each dataset category, the rows contain the number of different methods to be tested (MUTs) and the sum of unit tests written by experts. The superscript *cs* is used for C#, while *c++* stands for C++. The subscript *h* stands for the data set written by the experts, while *s* is the synthetically generated dataset.

Dataset	D	D^{cs}	D^{c++}	D_{train}	D_{eval}	D_h	D_s	D_h^{cs}	D_h^{c++}	D_s^{cs}	D_s^{c++}
MUTs	590	384	206	472	118	319	271	191	128	193	78
Unit tests	1027	434	593	822	205	726	301	268	458	166	135

context window sizes within the boundaries of the base model. The fine-tuning process was performed over 5 epochs for both models using the dataset in Table 1. At the tools level, we used the Hugging Face⁷ Transformers library and downloaded the base models publicly available on the platform. For connecting interfaces and the prompt, we have the LangChain⁸ APIs.

5.2. Quantitative Evaluation of generated tests

For each of the 590 MUTs in the source code dataset, we instruct the model to generate 5 unit tests, resulting in a total number of 2950 unit tests produced. In the experiments, we keep the same number of generated tests and average the results obtained over 10 independent samples.

Evaluation of correctness and utility. In line with previous studies on test generation based on learning ([16], [26], [28], [24]), we evaluate the precision of the generated tests based on two main criteria: (a) the compilation precision, which checks whether the test can be successfully compiled, and (b) the execution precision, which determines whether the test can pass execution (note that we are aware that each *MUT* is correct because it was selected to pass the tests). The comparative results between the two refined models and the three Code Llama-Instruct base models are shown in Table 2, before and after implementing our automatic post-generation corrections.

Table 2: A comparative analysis (in percentages) between the three CodeLlama Instruct base models without fine-tuning and the two proposed fine-tuned models. The evaluation of the correctness of the compilation is done after the first output of the model and follows the automatic heuristic that tries to fix some prevalent problems. The tests that pass the execution are then evaluated for coverage in the last two columns.

Model	Compilation initial		Compilation processed		Passing execution		Branch coverage		Statement coverage	
	C#	C++	C#	C++	C#	C++	C#	C++	C#	C++
CodeLlama 7B	26,6	22,5	28,5	24,2	18,2	13,1	-21,2	-19,1	-29,2	-31,5
CodeLlama 13B	29,6	21,7	32,6	27,1	20,9	18,3	-14,6	-15,3	-27,1	-26,0
CodeLlama 70B	33,4	24,8	38,1	29,4	22,4	21,1	+14,2	+15,1	-0,1	+1,7
GameUnitGen 7B	32,4	26,5	38,7	29,3	23,2	21,9	+17,4	+19,1	+4,3	+5,92
GameUnitGen 13B	37,3	31,9	53,5	42,7	28,4	22,0	+29,3	+33,87	+9,58	+13,0

The statement and branch coverages of the generated unit tests are evaluated by executing the tests in the same context used by the MUT. The last two columns in Table 2 show the overall improvements of the generated tests compared to the original set of expert-crafted unit tests available in $\mathcal{D}_l$ (i.e. a positive value means an increase in coverage). Note that in this evaluation we have disregarded the contribution of the expert dataset $\mathcal{D}_h$ in order to evaluate only the contribution of the generated unit tests.

In general, from both the correctness and coverage evaluation perspectives, we can deduce that the proposed techniques for automatically generating unit tests are not only beneficial, but can also be executed on users' systems from a computational perspective. The results shown in Table 2 indicate that fine-tuning on a user-defined task and dataset can help the smaller models get close and even outperform on some cases the larger models. This is also reported in the literature in related studies, such as: (a) training models

⁷ <https://huggingface.co/docs/transformers>

⁸ <https://www.langchain.com>

for code generation from general language models, [21], [11], or (b) fine-tuning models for interaction with external agents and tools [22], [22]. We assumed that the fine-tuning process would improve precision in both compilation and execution. This can be intuitively explained by the fact that the base LLM is further trained with game engine-specific functions and its output tokens are then associated with these examples rather than with general programming code without further clues. This practical aspect is important, as 7B and 13B can be used by consumer hardware with or without a GPU. On the Nvidia RTX 4090 GPU, for example, a similar experiment setup takes on average less than 2 minutes to create 5 tests with a total of 150 lines of code. The same results on a Core i9-13900k took 19 minutes. Compared to the base code Llama Instruct-70B, about nine times the GPU computing power is required, i.e. 140 GB of video memory, and it is almost impractical to wait for results with only a CPU, even if the machine had enough RAM.

5.3. Discussions and threats to validity

During the development of the proposed methods, several cycles were performed between the way the dataset $\mathcal{D}$ is collected, the fine-tuning process (including the setup and the parameters used) and the prompt engineering. Various factors in this context can affect the performance of the models. Different methods in the prompts or the representation of signature classes and functions, the division of requirements by bullets, or small punctuation details can play an important role in the quality of the response, especially with small-sized models. Another important observation made during the experiments is that it is usually better to use as little context as possible for the model to increase both the accuracy and correctness of compilation/execution. For example, instead of showing the entire code of a class, it is desirable to show only its interfaces. The evaluation of the models at test time is done exclusively with zero-shot prompting, as defined in Section 4.1. We have also experimented with few-shot prompting, but our current experiments have shown that including too much context in the model decreases the performance of the model instead of making it better adapted to the tasks in general. Considering these different parameters, we conclude that further fine-tuning of the methods for each user project might be needed to achieve the best results.

An *ablation* study on the used prompts, parameters, and examples of generated unit tests in comparison between the mentioned models and GPT3.5/GPT4 is provided as a supplement in our repository due to space limitations. The study also breaks down the compilation and execution classes by category and includes a human evaluation of the quality of the generated source code, i.e. how understandable it is.

6. Conclusion

In this study, we explored the possibilities of language learning models (LLMs) in the creation of unit test code for the games industry. Initial expectations and goals were set based on existing source code examples, commonly used game engines such as Unreal and Unity, and discussions with the game development industry. We summarize the following conclusions:

- Training smaller LLMs can provide better adaptability to the needs and requirements of game developers in terms of APIs for local infrastructures, private libraries, and standards.
- Improved computational effort for fine-tuning and creating unit tests at test time, as fine-tuned small LLMs can outperform their counterparts with large, generically targeted LLMs.
- Protection of the developer's source code and controllable costs through the ability to deploy everything locally.
- By observing various factors that can affect the quality of the results, we conclude that applying different post-processing heuristics can improve the compilation and execution rate of the generated tests. Although the methods can be applied immediately, we believe that depending on the applied project case and requirements, further fine-tuning might be needed to achieve the best possible performance.

In the future, we plan to focus on applying the human-in-the-loop concept to support the LLM model in improving and guiding the generation of unit tests.

Acknowledgements

This research is partially supported by the POCIDIF project in Action 1.2 "Romanian Hub for Artificial Intelligence" MySMIS no. 310483 and a grant of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI-UEFISCDI, project no. PN-IV-P8-8.1-PRE-HE-ORG-2023-0081, within PNCDI IV. We also thank our game development industry partners from Amber, Ubisoft, and Electronic Arts for their support.

References

- [1] Al-Ahmad, B., et al., 2021. Jacoco-coverage based statistical approach for ranking and selecting key classes in object-oriented software. *Journal of Engineering Science and Technology* 16, 3358–3386.
- [2] Alshahwan, N., et al., 2024. Automated unit test improvement using large language models at meta. 32nd ACM Symposium on the Foundations of Software Engineering (FSE 24) URL: <https://doi.org/10.48550/arXiv.2402.09171>.
- [3] Athiwaratkun, B., et al., 2023. Multi-lingual evaluation of code generation models. [arXiv:2210.14868](https://arxiv.org/abs/2210.14868).
- [4] Bender, E.M., et al., 2021. On the dangers of stochastic parrots: Can language models be too big?, in: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*.
- [5] Campos, J., et al., 2014. Continuous test generation: enhancing continuous integration with automated test generation, in: *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*.
- [6] Cassano, F., et al., 2023. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering* 49, 3675–3691. doi:[10.1109/TSE.2023.3267446](https://doi.org/10.1109/TSE.2023.3267446).
- [7] Chen, M., et al., 2021. Evaluating large language models trained on code [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- [8] Feng, Z., et al., 2020. CodeBERT: A pre-trained model for programming and natural languages, in: *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547.
- [9] Hu, E.J., et al., 2022. LoRA: Low-rank adaptation of large language models, in: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [10] Hui, L., Belkin, M., 2020. Evaluation of neural architectures trained with square loss vs cross-entropy in classification tasks. *arXiv preprint arXiv:2006.07322*.
- [11] Li, R., et al., 2023. Starcoder: may the source be with you! [arXiv:2305.06161](https://arxiv.org/abs/2305.06161).
- [12] Loshchilov, I., Hutter, F., 2019. Decoupled weight decay regularization, in: *7th International Conference on Learning Representations, ICLR 2019*.
- [13] Meister, C., Cotterell, R., 2021. Language model evaluation beyond perplexity, in: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*.
- [14] Moradi Dakhel, A., et al., 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software*.
- [15] Neelakanteswara, A., et al., 2024. RAGs to style: Personalizing LLMs with style embeddings, in: *Proceedings of the 1st Workshop on Personalization of Generative AI Systems*.
- [16] Nie, P., et al., 2023. Learning deep semantics for test completion, in: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 2111–2123. doi:[10.1109/ICSE48619.2023.00178](https://doi.org/10.1109/ICSE48619.2023.00178).
- [17] Nijkamp, E., et al., 2023. Codegen: An open large language model for code with multi-turn program synthesis. [arXiv:2203.13474](https://arxiv.org/abs/2203.13474).
- [18] Ouyang, L., et al., 2022. Training language models to follow instructions with human feedback. URL: <http://arxiv.org/abs/2203.02155>. cite arxiv:2203.02155.
- [19] Pearce, H., et al., 2022. Asleep at the keyboard? assessing the security of github copilot's code contributions, in: *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 754–768. doi:[10.1109/SP46214.2022.9833571](https://doi.org/10.1109/SP46214.2022.9833571).
- [20] Politowski, C., Petrillo, F., Guéhéneuc, Y.G., 2021. A survey of video game testing, in: *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*, pp. 90–99. doi:[10.1109/AST52587.2021.00018](https://doi.org/10.1109/AST52587.2021.00018).
- [21] Rozière, B., et al., 2024. Code llama: Open foundation models for code. [arXiv:2308.12950](https://arxiv.org/abs/2308.12950).
- [22] Schick, T.a., 2023. Toolformer: Language models can teach themselves to use tools, in: *Advances in Neural Information Processing Systems*.
- [23] Siddiq, M.L., et al., 2022. An empirical study of code smells in transformer-based code generation techniques, in: *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*.
- [24] Siddiq, M.L., et al., 2024. Using large language models to generate junit tests: An empirical study.
- [25] Straubinger, P., Fraser, G., 2023. A survey on what developers think about testing, in: *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 80–90. doi:[10.1109/ISSRE59848.2023.00075](https://doi.org/10.1109/ISSRE59848.2023.00075).
- [26] Tufano, M., et al., 2021. *arXiv*. URL: <https://www.microsoft.com/en-us/research/publication/unit-test-case-generation-with-transformers-and-focal-context/>.
- [27] Wang, Y., et al., 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.
- [28] Yuan, Z., et al., 2023. No more manual tests? evaluating and improving chatgpt for unit test generation. [arXiv:2305.04207](https://arxiv.org/abs/2305.04207).